IILM UNIVERSITY

Course Pack Framework

A Comprehensive Instructional Delivery Document

OBE
(Education)

What the Student should achieve?

OBLT
(Learning and Teaching)

How to make the students to achieve the Outcome

OBA
(Assessment)

How to measure what the students has



Prepared by: Ms.Daksha
Course Lead

School of Computer Science and Engineering

VISION

To be a globally recognized institute that fosters innovation, Critical thinking approach to create sustainable technological solutions and develop ethical, future-ready responsible professionals for the evolving digital society.

MISSION

M1. To provide quality education in Computer Science that equips students with strong theoretical foundations, practical skills, and the ability to solve real-world technological challenges using emerging technologies.

M2. To establishing state-of-the-art laboratories, research facilities and CoE that enable students and faculty to design, develop. and deploy innovative and sustainable engineering solutions.

M3. To cultivate a culture of innovation, creativity, and multidisciplinary research in emerging areas of computing to address global and societal challenges through establishment.

M4. To strengthen partnerships with industry, academic, and international collaborations to promote knowledge exchange, entrepreneurship, and impactful technological advancement.

Preamble

The Course Pack is a detailed pedagogical guide that outlines the components of instruction delivery by faculty members. It includes the course structure, course overview, objectives, prerequisite courses, program-specific outcomes (PSOs), course outcomes (COs), Bloom's Taxonomy (Knowledge Levels), types of courses, course articulation matrix, assessment patterns, course content, lesson plans, bibliography, problem-based learning or case studies, and student-centered learning (promoting self-learning and lifelong learning). The Course Pack ensures a consistent approach to course delivery across the university while allowing faculty members the flexibility to introduce innovative teaching methods and develop diverse assessment tools (including alternate assessment methods).

Each faculty member responsible for teaching a course develops the Course Pack. When multiple faculty members teach the same course, they collaborate as a team, with a senior faculty member (Course Lead) guiding the design and delivery of the course. This collaborative effort provides ample opportunities for faculty members to introduce innovative teaching strategies within the department or school.

Overall, the Course Pack serves as a comprehensive framework for teaching and learning, guiding faculty members across schools and departments in the university. It is not only a tool for evaluating student learning but also for continuously assessing the achievement of course learning outcomes for all students.

Scheme

The scheme provides an overview of work-integrated learning opportunities, offering students hands-on experience in real-world settings. It outlines what the course involves and the practical skills students will gain.

<table>
<tr><td>Course Title</td><td colspan="3">Data Structures & Algorithms</td><td colspan="3">Course Type</td><td colspan="3">Theory</td></tr>
<tr><td>Course Code</td><td colspan="3">26CSEC302</td><td colspan="3">Class</td><td colspan="3">B.Tech 2nd Year(3rd Sem)</td></tr>
<tr><td rowspan="6">Instruction Delivery</td><td>Activity</td><td>Credits</td><td>Weekly Hours</td><td colspan="3" rowspan="2">Total Number of Classes per Semester</td><td colspan="3" rowspan="2">Assessment in Weightage</td></tr>
<tr><td>Lecture</td><td>3</td><td>3</td></tr>
<tr><td>Tutorial</td><td>0</td><td>0</td><td rowspan="4">Lectures</td><td rowspan="4">Tutorial</td><td rowspan="4">Practical</td><td rowspan="4">CIA</td><td rowspan="4">MTE</td><td rowspan="4">ETE</td></tr>
<tr><td>Practical</td><td>4</td><td>4</td></tr>
<tr><td>Self- study</td><td>0</td><td>0</td></tr>
<tr><td>Total</td><td>7</td><td>7</td><td>45</td><td>0</td><td>30</td><td>0</td><td>0</td><td>0</td></tr>
<tr><td rowspan="3"></td><td>Course Lead</td><td colspan="2"></td><td colspan="3">Course Coordinator</td><td colspan="3"></td></tr>
<tr><td colspan="3">List of faculties teaching Theory</td><td colspan="6">List of faculties teaching Practical</td></tr>
<tr><td colspan="3">1.</td><td colspan="6">Not Applicable</td></tr>
</table>

Course Overview

Data Structures & Algorithms is a fundamental course in Computer Science and Engineering that develops students' ability to organize, process, and manipulate data efficiently and to design algorithms for solving computational problems. The course introduces the principles, properties, implementation, and applications of fundamental data structures such as arrays, linked lists, stacks, queues, trees, heaps, hash tables, and graphs. It also covers essential algorithmic techniques, including searching, sorting, recursion, divide-and-conquer, greedy methods, and graph-based algorithms.

Emphasis is placed on understanding the relationship between data structures and algorithmic efficiency. Students learn to analyze the time and space complexity of algorithms using asymptotic notations and to select appropriate data structures and algorithmic approaches for a given problem. Through theoretical concepts, implementation exercises, and problem-solving activities, the course aims to develop systematic computational thinking and the ability to design efficient and scalable solutions..

Pre- Requisites

Students should have:

- C Programming Language

Pre-Requisite course required	**C Programming Language**	
If, yes please fill in the Details	**Course Code**	**Course Title**
	NA	NA

Program Outcome (PO's)

Program Outcomes (POs) are typically defined by the relevant apex body or council. If a council does not prescribe specific POs, the school offering the program is responsible for designing and developing the POs in alignment with the Program Educational Objectives (PEOs). These POs must be reviewed and approved by the Board of Studies (BOS) and subsequently submitted to the Academic Council for final approval. Upon successfully completing the course, students will be able to:

PO No.	Description of the Program Outcome
PO1	**Engineering Knowledge:** Apply knowledge of mathematical logic, algorithmic principles, and computer science fundamentals to design and analyze data structures such as stacks, queues, linked lists, trees, graphs, and sorting/searching techniques.
PO2	**Problem Analysis:** analyze complex computational problems by identifying suitable data structures and evaluating their time–space complexity using asymptotic notation and algorithm analysis techniques.
PO3	**Design/Development of Solutions:** Design and develop efficient data structure–based solutions (binary trees, AVL trees, B-trees, priority queues, circular queues, graph structures, etc.) ensuring performance, reliability, and safety in real-world applications.
PO4	**Conduct Investigations of Complex Problems:** Conduct experiments on algorithm performance (sorting, searching, tree operations, graph traversals) using modelling, complexity analysis, and result interpretation to derive meaningful conclusions.
PO5	**Engineering Tool Usage:** Utilize programming tools, IDEs, debuggers, and algorithm visualization tools to implement and test data structures such as graphs, threaded trees, stacks, and hashing, recognizing tool limitations.

PO6	**The Engineer and The World:** Evaluate the societal and environmental impact of algorithm efficiency and data-structure choices, considering aspects like energy consumption, optimization, and sustainable software development.
PO7	**Ethics:** Apply ethical considerations while implementing and using data structures, ensuring data integrity, privacy, fairness in algorithms (like priority queues), and adherence to professional coding standards.
PO8	**Individual and Collaborative Team work:** Data-structure-based applications effectively both individually and in collaborative teams while contributing to shared coding tasks, documentation, analysis, and debugging.
PO9	**Communication:** Communicate technical ideas through algorithm reports, complexity charts, data structure diagrams (trees, graphs), and code documentation, using clear and professional language.
PO10	**Project Management and Finance:** Apply project management principles while developing data-structure projects—estimating resource usage, optimizing algorithm cost, managing timelines, and coordinating development tasks.
PO11	**Life-Long Learning:** Recognise the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

Program Specific Outcome (PSO's)

Program Specific Outcomes (PSOs) are concise statements outlining the abilities and competencies that graduates of a particular discipline-specific program are expected to achieve. Each program should define two to three PSOs.

PSO	Description of the Program Specific Outcome
PSO1	Graduates will be able design, implement, and analyze various data structures such as stacks, queues, linked lists, trees, and graphs, along with searching and sorting algorithms using appropriate programming techniques and complexity analysis.
PSO2	Graduates will be able to apply suitable algorithms and data structures to solve real-world computational problems efficiently by optimizing time and space complexity.
PSO3	Graduates will be able develop software solutions and applications using data structures and algorithms, utilizing modern programming tools and techniques for performance optimization and scalability.

Program Educational Objectives (PEOs)

PEO1: Graduates will apply strong foundations in computer science and emerging technologies to develop innovative and sustainable solutions to real-world problems**.**

PEO2: Graduates will pursue successful careers, higher education, professional certifications, and continuous learning to adapt to evolving technological advancements.

PEO3: Graduates will contribute to innovation, multidisciplinary teamwork, and industry–academia collaboration while addressing real-world and societal challenges through computing solutions.

PEO4: Graduates will demonstrate professional ethics, leadership, effective communication, and social responsibility, contributing to sustainable development and the global digital society.

.

Course Content

Course Objectives

The objective of the course is:

1. To introduce the fundamental concepts of data structures, including their representation, operations, and applications, enabling students to understand how data is organized and accessed efficiently.
2. To develop the ability to analyze algorithms using asymptotic notations, evaluating their time and space complexity for selecting the most efficient solution..
3. To provide knowledge and hands-on practice in implementing linear data structures such as arrays, stacks, queues, and linked lists, along with their operations and real-time applications.
4. To impart understanding of tree and graph structures, including binary trees, BST, AVL trees, threaded trees, B-trees, and graph traversal methods like BFS and DFS.
5. To teach various searching (linear, binary) and sorting algorithms (bubble, selection, quick, merge, heap) and help students compare their performance based on complexity analysis.
6. To enable students to select and apply suitable data structures and algorithms for solving computational problems effectively.

Course Content

UNIT 1:

Basic Terminologies: Elementary Data Organizations, Data Structure Operations: insertion, deletion, traversal etc.; Analysis of an Algorithm, Asymptotic Notations, Time-Space trade off.
Searching: Linear Search and Binary Search Techniques and their complexity analysis.

UNIT 2:

ADT Stack and its operations: Algorithms and their complexity analysis, Applications of Stacks: Expression Conversion and evaluation – corresponding algorithms and complexity analysis. ADT queue, Types of Queue: Simple Queue, Circular Queue, Priority Queue; Operations on each types of Queues: Algorithms and their analysis.

UNIT 3:

LinkedList: Singly linked lists: Representation in memory, Algorithms of several operations: Traversing, Searching, Insertion into, Deletion from linked list; Linked representation of Stack and Queue, Header nodes, Doubly linked list: operations on it and algorithmic analysis; Circular Linked Lists: All operations their algorithms and the complexity analysis.
Trees: Basic Tree Terminologies, Different types of Trees: Binary Tree, Threaded Binary Tree, Binary Search Tree, AVL Tree; Tree operations on each of the trees and their algorithms with complexity analysis. Applications of Binary Trees. B Tree, B+ Tree: definitions, algorithms and analysis.

UNIT 4:

Sorting: Objective and properties of different sorting algorithms: Selection Sort, Bubble Sort, Insertion Sort, Quick Sort, Merge Sort, Heap Sort; Performance and Comparison among all the methods, Hashing.

UNIT 5:

Graph : Basic Terminologies and Representations, Graph search and traversal algorithms and complexit y analysis.

UNIT 6:

Role of Data Structures in AI,State Space Search — BFS, DFS, IDS, Greedy, UCS with complexity table,Heuristic Functions & Priority Queues (A* deep dive), Knowledge Graphs, Decision Trees, K-D Trees, Data Structure Mapping (weights, DAG, attention), AI optimizing Data Structures, GNNs, RL for sorting,Full complexity comparison table for all AI algorithms

Course Outcome (CO)

Course Outcome	Upon successful completion of this course, the student will be able to:
CO1	Understand and analyze basic data structures, algorithms, asymptotic notations, and searching techniques.
CO2	Implement and evaluate linear data structures such as stacks and queues along with expression evaluation.
CO3	Apply and analyze linked lists, trees, and advanced tree structures for problem-solving.
CO4	Implement and compare various sorting and hashing techniques based on performance.
CO5	Use graph representations and traversal algorithms to solve complex computational problems.

Course Articulation Matrix

The course articulation matrix represents the relationship between Course Outcomes, Program Outcomes and Program Specific Outcome, highlighting the expected strength of their alignment at three levels: low, medium, and high.

COs	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO1	3	3	1	2	2	1	-	-	-	-	-	3	2	1
CO2	3	2	2	2	2			-	-	-	-	3	3	2
CO3	3	3	2	3	3	1	-	-	-	-	-	3	3	3
CO4	3	3	2	2	3	-	-	-	1	-	-	2	3	3
CO5	3	3	2	3	3	1	-	-	-	-	-	2	3	3

3: High Correlation | 2: Medium Correlation | 1: Low Correlation |-: No Correlation

Taxonomy Level of the Course Outcomes

Bloom's taxonomy is a hierarchical framework used to classify educational learning objectives based on their complexity and specificity. It encompasses three primary learning domains: cognitive, affective, and psychomotor.

CO No.	Remember KL1	Understand KL 2	Apply KL 3	Analyse KL 4	Evaluate KL 5	Create KL 6
CO1	Yes	Yes	Yes			
CO2	Yes	Yes	Yes			
CO3	Yes	Yes	Yes			
CO4			Yes	Yes	Yes	
CO5			Yes	Yes	Yes	Yes

Evaluation Matrix:

Theory

- **Continuous Internal Assessment (CIA)**
  Includes assignments and quizzes to evaluate continuous learning during the semester.
- **Mid Term Examination (MTE)**
  Mid Term examination is conducted in the middle of the semester to assess the understanding of course concepts and theoretical knowledge.
- **End Term Examination (ETE)**
  Final written examination conducted at the end of the semester to assess the overall understanding of course concepts and theoretical knowledge.

Assessment Component	Category	Marks	Weightage (%)
Attendance	Internal	5	5%
Professional Conduct/Learning Attitude	Internal	5	5%
Certification	Internal	10	10%
Viva / Presentation	Internal	10	10%
Continuous Assessment Total		30	30%
Mid-Term Test (MT)	MT	20	20%
Total (Internal)		50	50%
End-Term Examination (ETE)	ET	50	50%
ET Total		50	50%
Grand Total		100	100%

LAB Evaluation Matrix

Not Applicable

Lecture Plan

Lec No.	Topic	Skills Developed	Competency
1	Introduction to Data Structures	Understanding concepts	Ability to classify data structures
2	Abstract Data Types (ADT)	Logical modelling	Create ADT definitions
3	Algorithm Analysis & Asymptotic Notations	Analytical skills	Evaluate time & space complexity
4	Complexity Examples & Practice	Problem solving	Apply Big-O to real code
5	Linear & Binary Search	Coding & logic	Implement basic search algorithms
6	Recursion Basics	Recursive thinking	Apply recursion in problems
7	Stack ADT, Representation	Implementation skills	Build stack using array
8	Stack Operations	Debugging	Perform push/pop operations
9	Applications of Stack	Analytical	Solve expression problems
11	Infix, Prefix, Postfix Concepts	Expression handling	Convert expressions
12	Expression Evaluation	Program logic	Evaluate postfix/prefix expressions
13	Queue ADT	Implementation	Create linear queue
14	Circular Queue	Memory management	Implement circular queue logic
15	Priority Queue	Understanding priority	Use heap-based priority queues
16	Deque Operations	Multi-operation handling	Implement front/back operations
17	Singly Linked List Ops	Dynamic memory skills	Insert/delete in SLL
18	Doubly Linked List Ops	Bidirectional logic	Implement DLL operations
19	Circular Linked List	Circular logic	Build CLL
20	Applications of Linked Lists	Real-world thinking	Use lists in applications
21	Introduction to Trees	Structural understanding	Identify tree components
22	Binary Tree Representation	Logical modelling	Build binary tree nodes
23	Tree Traversals (Inorder, Preorder, Postorder)	Algorithmic thinking	Perform all traversals
24	Binary Search Tree (BST)	Problem solving	Implement search/insert/delete
25	AVL Trees: Rotations	Critical thinking	Apply LL, RR, LR, RL rotations
26	AVL Tree Insertion/Deletion	Advanced coding	Maintain balanced trees
27	B-Trees Introduction	Large-data handling	Understand multi-way trees
28	B-Tree Operations	Analytical	Apply split/merge operations
29	B+ Trees Introduction	Data indexing	Understand search-optimized trees
30	Hashing Concepts	Mapping skills	Apply hashing functions
31	Collision Resolution Techniques	Problem solving	Implement chaining & probing
32	Sorting – Bubble, Selection	Fundamental coding	Compare basic sorting
33	Insertion Sort, Shell Sort	Logic development	Analyze improvement over simple sorts
34	Merge Sort	Divide & conquer	Implement recursive merge sort
35	Quick Sort	Partitioning logic	Apply quick sort steps
36	Heap & Heap Sort	Priority handling	Implement max/min heaps
37	Introduction to Graphs	Structural mapping	Represent graphs
38	Graph Representation (Matrix/List)	Coding skills	Choose suitable representation
39	BFS Algorithm	Traversal skills	Implement BFS
40	DFS Algorithm	Problem solving	Implement DFS
41	Shortest Path (Dijkstra)	Analytical	Compute weighted shortest path
42	Minimum Spanning Trees: Prim's Algorithm	Optimization	Apply MST logic
43	Kruskal's Algorithm	Sorting + graph logic	Apply union–find structures



44	Applied Graph Problems	Real-world skills	Solve path, network, connectivity problems
45	Course Revision & Problem Solving	Consolidated skills	Integrate all DSA concepts



List of Lab Exercises

NOT APPLICABLE FOR THIS COURSE

Reference's

TEXT BOOKS:

1. Reema Thareja, Data Structures Using C, 2nd Edition, Oxford University Press, Reprints: 2017–2023.
2. Mark Allen Weiss, Data Structures and Algorithm Analysis in C, 4th Edition, Pearson Education, Reprints: 2015–2022.
3. Ellis Horowitz, Sartaj Sahni, Susan Anderson-Freed, Fundamentals of Data Structures in C, 2nd Edition, Silicon Press, Reprints: 2011–2022.
4. Yashavant Kanetkar, Data Structures Through C, Revised Edition, BPB Publications, Reprints: 2015–2024

REFERENCE BOOKS:

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, Introduction to Algorithms, 4th Edition, MIT Press, Reprints: 2020–2024.
2. Robert Sedgewick, Kevin Wayne, Algorithms, 4th Edition, Addison-Wesley, Reprints: 2014–2023.
3. Michael T. Goodrich, Roberto Tamassia, David M. Mount, Data Structures and Algorithms in C, 2nd Edition, Wiley, Reprints: 2011–2022.
4. Michael T. Goodrich, Roberto Tamassia, David M. Mount, Data Structures and Algorithms in C, 2nd Edition, Wiley, Reprints: 2011–2022.
5. Narasimha Karumanchi, Data Structures and Algorithms Made Easy, 5th Edition, CareerMonk Publications, Reprints: 2016–2023.

MooC Course:

Coursera –

https://www.coursera.org/learn/bits-data-structures-and-algorithms

NPTEL:

https://nptel.ac.in/courses/106102064

Problem-Based Learning / Case Studies:

- Students study the components of a DSA using game engine and design a simple architecture for a 2D adventure game including input, rendering, and game state management using Unity software.
- Students apply concepts such as velocity, acceleration, and gravity to simulate motion and interactions in a simple game using unity and Figma tool.

Benchmarking 1 (Premier Indian Institution/University Syllabus):

Name of University/ Institution

The syllabus of the course is benchmarked with the Indian Institute of Information Technology Nagpur, where similar courses on OOPS analysis and Design is taught.

Benchmarking 2 (International Institution/University Syllabus):

Name of University/ Institution

The course syllabus is benchmarked with programs offered by The American College of Greece.

Benchmarking 3 (Private Institution/University Syllabus)

Name of University/ Institution

The course syllabus is also benchmarked with reputed private universities, such as Chandigarh University Punjab.

Comparison of Benchmarked Syllabus (% of Similarity)

Sr. No.	Module / Topic	Benchmark 1 (Savitribai Phule Pune University)	Benchmark 2 (Gujarat Technological University)	Benchmark 3 (Anna University)	ABET	IEEE	Syllabus of IILM	% of Similarity
1	Basic Terminologies, Data Organization, Algorithm Analysis	85%	80%	82%	90%	88%	Included	85%
2	Searching Techniques (Linear & Binary Search)	88%	85%	87%	90%	89%	Included	88%

Sr. No.	Module / Topic	Benchmark 1 (Savitribai Phule Pune University)	Benchmark 2 (Gujarat Technological University)	Benchmark 3 (Anna University)	ABET	IEEE	Syllabus of IILM	% of Similarity
3	Stack ADT and Applications	90%	88%	85%	92%	90%	Included	89%
4	Queue ADT (Simple, Circular, Priority Queue)	87%	86%	85%	90%	88%	Included	87%
5	Linked List (Singly, Doubly, Circular)	89%	90%	88%	92%	90%	Included	90%
6	Tree Concepts (Binary Tree, BST, AVL, Threaded Tree)	85%	87%	86%	90%	89%	Included	87%
7	B-Tree and B+ Tree	82%	80%	84%	85%	86%	Included	83%
8	Sorting Algorithms (Bubble, Selection, Insertion, Quick, Merge, Heap)	90%	92%	89%	95%	93%	Included	92%
9	Hashing	80%	82%	78%	85%	84%	Included	82%
10	Graph Representation and Traversal	88%	87%	86%	90%	91%	Included	88%
Overall Similarity	**Complete Data Structures Syllabus**	**86%**	**86%**	**85%**	**91%**	**89%**	**Fully Covered**	**87%**

QUESTION BANK (300 Questions)

50 Questions from Each Unit

(50*6 = 300 Questions)

Ques. No.	CO's	Answer Length	Marks	BKL	Statement of the Question
1	1	Short	2	K1	Define data structure.
2	1	Short	2	K1	Differentiate between linear and non-linear data structures.
3	1	Short	2	K1	What is an algorithm?
4	1	Short	2	K1	Explain stack operations with example
5	1	Short	2	K1	Explain BST operations with examples.
6	1	Short	2	K1	What is Big-O notation?
7	1	Short	2	K1	Define asymptotic analysis.
8	1	Short	2	K1	What is the best-case complexity?
9	1	Short	2	K1	What is the worst-case complexity?
10	1	Short	2	K1	Define average-case complexity.
11	1	Short	2	K1	What is recursion?
12	1	Short	2	K1	State two applications of recursion.
13	1	Short	2	K1	Define array.
14	1	Short	2	K1	What are the advantages of arrays?
15	1	Short	2	K1	State the limitation of arrays.
16	1	Short	2	K1	Define linear search.
17	1	Short	2	K1	Define binary search.
18	1	Short	2	K1	What is the prerequisite for binary search?
19	1	Short	2	K1	Define algorithm efficiency.
20	1	Short	2	K1	What is dynamic memory allocation?
21	1	Short	2	K1	Define abstract data type (ADT).
22	1	Short	2	K1	What is the difference between compile-time and run-time memory allocation?
23	1	Short	2	K1	Define complexity analysis.
24	1	Short	2	K1	What is logarithmic complexity?
25	1	Short	2	K1	Define polynomial complexity.
26	1	Short	2	K1	What is recursion stack?
27	1	Short	2	K1	What is tail recursion?
28	1	Short	2	K1	Differentiate iterative and recursive methods.
29	1	Short	2	K1	Define sparse matrix.
30	1	Short	2	K1	State two applications of arrays.
31	1	Short	2	K1	What is matrix representation?
32	1	Short	2	K1	What is searching?
33	1	Medium	6	K2	Explain types of data structures with examples.

34	1	Medium	6	K2	Compare linear and non-linear data structures.
35	1	Medium	6	K2	Explain algorithm analysis with examples.
36	1	Medium	6	K2	Differentiate time and space complexity.
37	1	Medium	6	K2	Explain Big-O notation with examples.
38	1	Medium	6	K2	Analyze complexity of linear search.
39	1	Medium	6	K2	Explain binary search algorithm.
40	1	Medium	6	K2	Compare linear and binary search.
41	1	Medium	6	K2	Explain recursion with example.
42	1	Medium	6	K2	Discuss applications of recursion.
43	1	Medium	6	K2	Explain array operations.
44	1	Medium	6	K2	Write algorithm for matrix addition.
45	1	Medium	6	K2	Explain multidimensional arrays.
46	1	Medium	6	K2	Discuss advantages and disadvantages of arrays.
47	1	Medium	6	K2	Explain sparse matrix representation.
48	1	Medium	6	K2	Write algorithm for factorial using recursion.
49	1	Medium	6	K2	Explain iterative and recursive approaches.
50	1	Medium	6	K2	Discuss algorithm efficiency.
51	1	Medium	6	K2	Explain asymptotic notations.
52	1	Medium	6	K2	Discuss memory allocation techniques.
53	1	Medium	6	K2	Explain dynamic memory allocation.
54	1	Medium	6	K2	Explain abstract data type.
55	1	Medium	6	K2	Compare static and dynamic data structures.
56	1	Medium	6	K2	Explain pseudocode design.
57	1	Medium	6	K2	Discuss divide-and-conquer approach.
58	1	Medium	6	K2	Explain recursive tree method.
59	1	Medium	6	K2	Discuss algorithm optimization methods.
60	1	Medium	10	K2	Explain asymptotic notations in detail with suitable examples.
61	1	Medium	10	K3	Discuss algorithm analysis and explain time and space complexity.
62	1	Medium	10	K3	Compare linear search and binary search with algorithms and complexity analysis.
63	1	Medium	10	K3	Explain the binary search algorithm with a step-by-step example.
64	1	Medium	10	K3	Describe elementary data organizations and operations in detail.
65	1	Medium	10	K6	Discuss the concept of time-space tradeoff with applications.
66	1	Medium	10	K3	Explain best case, average case, and worst case analysis with examples.
67	1	Medium	10	K3	Write and explain algorithms for insertion, deletion, and traversal operations.
68	1	Medium	10	K2	Explain recursive algorithms and compare them with iterative algorithms.
69	1	Medium	10	K3	Discuss the role of data structures in efficient algorithm design.
70	1	Medium	10	K3	Explain the limitations of linear search and advantages of binary search.
71	1	Medium	10	K2	Analyze searching techniques based on complexity and performance.
72	2	Short	2	K1	Define stack.
73	2	Short	2	K1	What is LIFO?
74	2	Short	2	K1	State two applications of stack.
75	2	Short	2	K1	Define push operation.
76	2	Short	2	K1	Define pop operation.
77	2	Short	2	K1	What is stack overflow?
78	2	Short	2	K1	What is stack underflow?
79	2	Short	2	K1	Define queue.
80	2	Short	2	K1	What is FIFO?
81	2	Short	2	K1	Define enqueue operation.
82	2	Short	2	K1	Define dequeue operation.
83	2	Short	2	K1	What is circular queue?
84	2	Short	2	K1	Define priority queue.
85	2	Short	2	K1	What is deque?
86	2	Short	2	K1	Define linked list.
87	2	Short	2	K1	What is a node in linked list?
88	2	Short	2	K1	Define singly linked list.
89	2	Short	2	K1	Define doubly linked list.
90	2	Short	2	K1	Define circular linked list.
91	2	Short	2	K2	State advantages of linked list.
92	2	Short	2	K2	State disadvantages of linked list.
93	2	Short	2	K2	What is dynamic data structure?

94	2	Short	2	K2	Define self-referential structure.
95	2	Short	2	K2	What is memory fragmentation?
96	2	Short	2	K2	What is pointer?
97	2	Short	2	K2	Define circular linked list.
98	2	Short	2	K2	State advantages of linked list.
99	2	Short	2	K2	State disadvantages of linked list.
100	2	Short	2	K2	What is dynamic data structure?
101	2	Short	2	K2	Define self-referential structure.
102	2	Short	2	K2	What is memory fragmentation?
103	2	Short	2	K2	What is pointer?
104	2	Medium	6	K3	Explain stack operations with example.
105	2	Medium	6	K3	Implement stack using array.
106	2	Medium	6	K3	Explain stack applications.
107	2	Medium	6	K3	Convert infix to postfix expression.
108	2	Medium	6	K3	Evaluate postfix expression using stack.
109	2	Medium	6	K3	Explain recursion using stack.
110	2	Medium	6	K3	Explain queue operations.
111	2	Medium	6	K3	Implement queue using array.
112	2	Medium	6	K2	Explain circular queue with example.
113	2	Medium	6	K3	Compare linear queue and circular queue.
114	2	Medium	6	K3	Explain priority queue.
115	2	Medium	6	K3	Explain deque operations.
116	2	Medium	6	K3	Implement singly linked list.
117	2	Medium	6	K3	Explain doubly linked list operations.
118	2	Medium	6	K3	Discuss circular linked list.
119	2	Medium	6	K3	Compare singly and doubly linked lists.
120	2	Medium	6	K3	Explain insertion in linked list.
121	2	Long	6	K6	Explain deletion in linked list.
122	2	Medium	6	K3	Reverse a linked list.
123	2	Medium	6	K3	Explain polynomial addition using linked list.
124	2	Medium	6	K3	Discuss linked implementation of stack.
125	2	Medium	6	K3	Discuss linked implementation of queue.
126	2	Medium	6	K3	Explain applications of linked list.
127	2	Medium	6	K3	Explain memory management in linked list.
128	2	Medium	6	K3	Compare stack and queue.
129	2	Medium	6	K3	Discuss queue applications.
130	2	Medium	6	K3	Explain stack operations with example.
131	2	Long	10	K3	Explain ADT Stack in detail with algorithms and complexity analysis.
132	2	Long	10	K3	Discuss applications of stack in expression conversion and evaluation.
133	2	Long	10	K3	Explain infix to postfix conversion with algorithm and example.
134	2	Long	10	K3	Write algorithms for stack operations and analyze their complexity.
135	2	Long	10	K2	Explain queue ADT and its operations with suitable examples.
136	2	Long	10	K6	Compare simple queue, circular queue, and priority queue.
137	2	Long	10	K6	Explain circular queue implementation with algorithm.
138	2	Long	10	K6	Discuss priority queue and its real-world applications.
139	2	Long	10	K6	Explain stack implementation using arrays and linked lists.
140	2	Long	10	K6	Explain queue implementation using arrays and linked lists.
141	2	Long	10	K3	Describe postfix expression evaluation using stack with example.
142	2	Long	10	K6	Compare stack and queue with suitable applications.
143	3	Short	2	K1	Define tree.
144	3	Short	2	K1	What is root node?
145	3	Short	2	K1	Define leaf node.

146	3	Short	2	K1	What is binary tree?
147	3	Short	2	K1	Define full binary tree.
148	3	Short	2	K1	Define complete binary tree.
149	3	Short	2	K1	What is binary search tree?
150	3	Short	2	K1	Define AVL tree.
151	3	Short	2	K1	What is tree traversal?
152	3	Short	2	K1	Define inorder traversal.
153	3	Short	2	K1	Define preorder traversal.
154	3	Short	2	K1	Define postorder traversal.
155	3	Short	2	K1	Define graph.
156	3	Short	2	K1	What is vertex in graph?
157	3	Short	2	K1	Define edge.
158	3	Short	2	K1	What is directed graph?
159	3	Short	2	K1	Define undirected graph.
160	3	Short	2	K2	What is weighted graph?
161	3	Short	2	K2	Define connected graph.
162	3	Short	2	K2	What is cyclic graph?
163	3	Short	2	K2	Define degree of vertex.
164	3	Short	2	K2	What is path in graph?
165	3	Short	2	K2	Define adjacency matrix.
166	3	Short	2	K2	Define adjacency list.
167	3	Short	2	K2	What is BFS?
168	3	Short	2	K2	Define DFS.
169	3	Short	2	K2	What is spanning tree?
170	3	Short	2	K2	Define minimum spanning tree.
171	3	Short	2	K2	What is heap?
172	3	Short	2	K2	Define max heap.
173	3	Short	2	K2	Define min heap.
174	3	Short	2	K2	What is expression tree?
175	3	Medium	6	K3	Define singly linked list and explain its representation in memory.
176	3	Medium	6	K3	Write an algorithm for traversal in singly linked list.
177	3	Medium	6	K3	Explain insertion operation in singly linked list.
178	3	Medium	6	K3	Describe deletion operation in singly linked list.
179	3	Medium	6	K3	Explain searching in linked list.
180	3	Medium	6	K3	Differentiate between singly and doubly linked lists.
181	3	Medium	6	K3	Explain doubly linked list operations.
182	3	Medium	6	K3	Discuss circular linked list with suitable examples.
183	3	Medium	6	K4	Explain header nodes in linked lists.
184	3	Medium	6	K4	Describe linked representation of stack and queue.
185	3	Medium	6	K4	Define tree and explain basic tree terminologies.
186	3	Medium	6	K4	Differentiate between binary tree and binary search tree.
187	3	Medium	6	K4	Explain threaded binary tree.
188	3	Medium	6	K4	Discuss AVL tree and its properties.
189	3	Medium	6	K4	Explain tree traversals with examples.
190	3	Medium	6	K4	Describe applications of binary trees.
191	3	Medium	6	K4	Explain B-tree and its properties.
192	3	Medium	6	K4	Discuss B+ tree and its applications.
193	3	Medium	6	K4	Compare AVL tree and binary search tree.
194	3	Medium	6	K4	Explain operations on binary search tree.
195	3	Medium	6	K4	Define singly linked list and explain its representation in memory.
196	3	Medium	6	K4	Explain the concept of balanced trees.
197	3	Medium	6	K4	Discuss applications of AVL trees in databases.
198	3	Medium	6	K5	Explain insertion at beginning, middle, and end in singly linked list.
199	3	Medium	6	K5	Discuss deletion of a node in doubly linked list with example.
200	3	Medium	6	K5	Explain advantages and disadvantages of circular linked list.
201	3	Medium	6	K5	Describe preorder, inorder, and postorder tree traversals.

202	3	Long	10	K3	Explain singly linked list operations with algorithms.
203	3	Long	10	K6	Discuss insertion and deletion operations in doubly linked list.
204	3	Long	10	K3	Explain circular linked list and its operations with examples.
205	3	Long	10	K3	Compare singly, doubly, and circular linked lists.
206	3	Long	10	K6	Explain representation and traversal of binary trees.
207	3	Long	10	K6	Discuss binary search tree operations with suitable examples.
208	3	Long	10	K6	Explain AVL tree insertion and balancing techniques.
209	3	Long	10	K3	Describe threaded binary tree with advantages.
210	3	Long	10	K6	Explain tree traversal algorithms in detail.
211	3	Long	10	K6	Discuss applications of binary trees in computer science.
212	3	Long	10	K5	Explain linked representation of stack and queue.
213	3	Short	2	K1	Define bubble sort.
214	4	Short	2	K2	Define selection sort.
215	4	Short	2	K2	Define insertion sort.
216	4	Short	2	K2	What is merge sort?
217	4	Short	2	K2	Define quick sort.
218	4	Short	2	K1	What is heap sort?
219	4	Short	2	K2	Define radix sort.
220	4	Short	2	K1	Define counting sort.
221	4	Short	2	K2	What is stable sorting?
222	4	Short	2	K1	Define unstable sorting.
223	4	Short	2	K1	What is in-place sorting?
224	4	Short	2	K2	Define divide-and-conquer sorting.
225	4	Short	2	K2	What is pivot element?
226	4	Short	2	K1	Define partitioning.
227	4	Short	2	K2	State best-case complexity of quick sort.
228	4	Short	2	K2	State worst-case complexity of quick sort.
229	4	Short	2	K2	Define merge operation.
230	4	Short	2	K1	What is heapify?
231	4	Short	2	K1	Define comparison sorting.
232	4	Short	2	K2	What is non-comparison sorting?
233	4	Short	2	K2	Define searching algorithm.
234	4	Short	2	K2	State complexity of binary search.
235	4	Short	2	K2	State complexity of linear search.
236	4	Short	2	K2	Define interpolation search.
237	4	Short	2	K2	What is hashing?
238	4	Short	2	K2	Define hash function.
239	4	Short	2	K2	What is collision?
240	4	Short	2	K2	Define chaining.
241	4	Short	2	K1	What is open addressing?
242	4	Short	2	K1	Define rehashing.
243	4	Short	2	K1	What is load factor?
244	4	Short	2	K1	Define bucket hashing.
245	4	Medium	6	K4	Explain the objectives and properties of sorting algorithms.
246	4	Medium	6	K4	Describe selection sort with an example.
247	4	Medium	6	K4	Explain bubble sort algorithm.
248	4	Medium	6	K4	Discuss insertion sort with suitable example.
249	4	Medium	6	K4	Explain quick sort technique.
250	4	Medium	6	K4	Describe merge sort algorithm.
251	4	Medium	6	K4	Explain heap sort with example.
252	4	Medium	6	K4	Compare stable and unstable sorting algorithms.

253	4	Medium	6	K4	Discuss internal and external sorting.
254	4	Medium	6	K4	Explain the complexity of bubble sort.
255	4	Medium	6	K4	Compare selection sort and insertion sort.
256	4	Medium	6	K4	Explain divide and conquer strategy in merge sort.
257	4	Medium	6	K4	Discuss partitioning process in quick sort.
258	4	Medium	6	K4	Explain heap data structure used in heap sort.
259	4	Medium	6	K4	Compare merge sort and quick sort.
260	4	Short	2	K1	Define B-tree.
261	4	Short	2	K1	Define B+ tree.
262	4	Short	2	K1	What is trie?
263	4	Short	2	K1	Define red-black tree.
264	4	Short	2	K1	What is disjoint set?
265	4	Short	2	K1	Define union-find algorithm.
266	4	Short	2	K1	What is shortest path algorithm?
267	4	Medium	6	K2	Explain best case and worst case complexity of sorting algorithms.
268	4	Medium	6	K2	Define hashing and explain its importance.
269	4	Medium	6	K2	Discuss hash functions with examples.
270	4	Medium	6	K2	Explain collision handling techniques in hashing.
271	4	Medium	6	K2	Compare linear probing and chaining methods.
272	4	Long	10	K6	Explain selection sort with algorithm and complexity analysis.
273	4	Long	10	K3	Discuss bubble sort and insertion sort with examples.
274	4	Long	10	K6	Explain merge sort using divide and conquer approach.
275	4	Long	10	K6	Write quick sort algorithm and analyze its complexity.
276	4	Long	10	K3	Explain heap sort with suitable example and complexity analysis.
277	4	Long	10	K6	Compare all sorting algorithms based on performance.
278	4	Long	10	K5	Discuss advantages and disadvantages of different sorting methods.
279	4	Long	10	K5	Explain hashing techniques and collision resolution methods.
280	4	Long	10	K5	Compare merge sort, quick sort, and heap sort.
281	4	Long	10	K5	Explain stable and unstable sorting algorithms in detail.
282	4	Long	10	K5	Discuss the role of sorting in data processing applications.
283	4	Long	10	K5	Explain hash table implementation with examples.
284	4	Long	10	K5	Describe linear probing, quadratic probing, and chaining.
285	5	Medium	6	K5	Define graph and explain its basic terminologies.
286	5	Medium	6	K5	Differentiate between directed and undirected graphs.
287	5	Medium	6	K5	Explain graph representation using adjacency matrix.
288	5	Medium	6	K5	Describe adjacency list representation of graphs.
289	5	Medium	6	K5	Explain graph traversal techniques.
290	5	Medium	6	K5	Discuss Breadth First Search (BFS) algorithm.
291	5	Medium	6	K5	Explain Depth First Search (DFS) algorithm.
292	5	Medium	6	K5	Compare BFS and DFS traversal methods.
293	5	Long	10	K6	Explain cyclic and acyclic graphs.
294	5	Long	10	K6	Describe connected and disconnected graphs.
295	5	Long	10	K6	Explain graph traversal complexity analysis.
296	5	Long	10	K6	Discuss real-life applications of graphs.
297	5	Long	10	K6	Explain spanning tree with example.
298	5	Long	10	K6	Describe graph searching techniques.
299	5	Long	10	K6	Explain advantages of adjacency list over adjacency matrix.
300	5	Long	10	K6	Discuss graph representation in memory.

IILM UNIVERSITY



SUSTAINABLE DEVELOPMENT GOALS

IILM University

Plot No. 16-18, Knowledge Park – II,

Greater Noida, Uttar Pradesh 201306

https://iilm.ac.in